

O/D-Conditional Graph Diffusion with Learned $\langle \text{end} \rangle$ Termination — Experimental Results

Status (2026-07-14, final for the full-canvas track). All results in this report are final: the base d-sweep (§2–3) and the A1+A3 / A1+A2+A3 / A1+A3+A4 ablations (§4). Best configuration: **A1+A2+A3 at CFG $w = 1.5$ (raw arrival 0.077, refined 0.335)**. The block-diffusion stage (semi-AR generation targeting the long-horizon failure mode of §4) is training and will be reported separately in [RESULTS_BD.pdf](#).

Data & model: Porto taxi trajectories on the shrunk road network (1,390 vertices, 2,002 undirected edges), 1.93M paths (`porto_v3_normal` ; the v3-0.05/v3-0.1 variants are verified identical); path length min 2 / mean 23.2 / max 59. Full-canvas discrete diffusion with the repo's uniform-state graph CTMC kernel $Q_t = \exp((A-D) \cdot \beta_t)$ ($T = 100, \beta \in [1e-4, 10]$), conv-UNet denoiser `EPSM_OD_EOS` (dims [100,120,200], node2vec embeddings, 2.81M params), 1 epoch = 48.2k iters (bs 32, lr $5e-4$), ≈ 1.9 h per model on one RTX 3090.

Length handling (LLaDA-style, no external length model): every path is padded to a fixed 64-token canvas with an $\langle \text{end} \rangle$ state the model predicts like any other state; because the kernel is a graph CTMC, $\langle \text{end} \rangle$ is a CTMC *state* — a virtual node of total degree d , uniformly connected (§7.2). Sampling truncates at the first $\langle \text{end} \rangle$. The original GMM length model is removed entirely.

Metrics (1,000 real OD pairs, seed 1): **valid** = every consecutive edge exists; **arrival** = endpoint equals the real destination; **valid $\wedge$ arr** = both (headline usable-path metric); **hit** = destination appears anywhere; EM/PC as in `eval_shortest.py`; **len err** = mean $|\text{len}(\text{gen}) - \text{len}(\text{real})|$; **len corr** = Pearson $\text{corr}(\text{len}(\text{gen}_i), \text{len}(\text{real}_i))$ over OD pairs — whether the emergent length *tracks per-pair variation* (scale-free; absolute calibration is len err / mean length). Variants scored **raw** and after **refine_es** (`eval_shortest.refine()`). Raw validity is ≈ 1.0 *by construction* (adjacency-constrained decode), so arrival is the discriminating column.

1. How the condition enters the architecture

EPSM_OD_EOS — pipeline (top) and the injection point of c inside each block (inset, bottom)

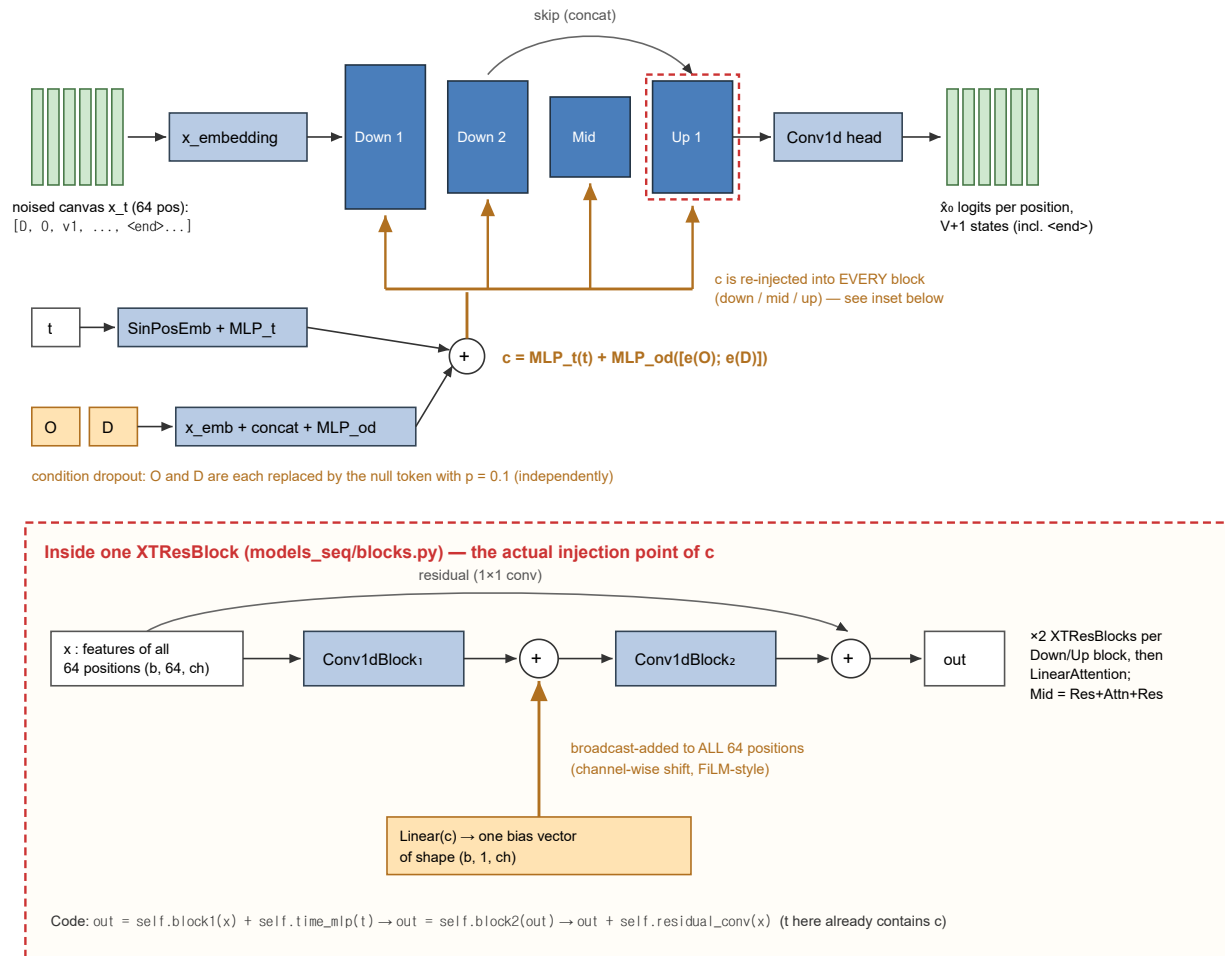


Figure 1. Architecture redrawn in the style of the reference paper's Figure 4 (ICLR'24 graph-constrained diffusion, whose EPSM this repo implements). **Top:** the noised canvas is embedded per position and passes through a 1-D conv trunk (no down/up-sampling, exactly as in the reference — the "Down/Up" names refer only to channel widths — plus a skip concatenation) into a Conv1d head that outputs per-position $\hat{x}_0$ logits over $V+1$ states. The condition c is the sum of the time embedding and the O/D embedding. **Inset —** how c acts inside the UNet: in every XTResBlock, c passes through that block's own `Linear` layer, producing one channel-wise bias vector per sample which is broadcast-added to the features of all 64 positions between the two convolutions (a FiLM-style shift; the same c is re-injected at every block and depth, so no layer can "forget" the condition). Note this modulation is position-uniform — it biases which vertices/channels are globally favored but carries no positional information, which is exactly why the in-context destination token (A3) adds steering power the embedding channel cannot: the dst token occupies a specific canvas position and interacts with neighboring positions through convolution and attention.

1.1 Where this lives in the code

Global conditioning channel — `models_seq/eps_models.py`, `EPSM_00`:

```
self.od_mlp = nn.Sequential(
    nn.Linear(2 * emb_dim, 4 * time_dim, device=device),
    nn.Mish(),
    nn.Linear(4 * time_dim, time_dim, device=device),
)

def forward(self, xt_padded, lengths, t, od=None):
    t = self.time_mlp(t)
    if od is None:
        # unconditional: null tokens
        od = torch.full((xt_padded.shape[0], 2), self.null_idx, ...)
    od_emb = rearrange(self.x_embedding(od), "b n e -> b (n e)")
    t = t + self.od_mlp(od_emb)
    # condition merged into the time pathway
    ...
```

Per-block injection point — `models_seq/blocks.py`, `XTResBlock.forward` (pre-existing; the condition rides on `t`):

```
out = self.block1(x) + self.time_mlp(t) # Conv1d(x) + Linear(c), broadcast over positions
out = self.block2(out)
return out + self.residual_conv(x)
```

Condition extraction + dropout (training) — `models_seq/seq_models.py`, `Restorer.forward`:

```
od = torch.stack([torch.stack([x[0], x[-1]]) for x in raw_xs]).long()
if self.training and self.od_dropout > 0:
    drop = torch.rand(od.shape) < self.od_dropout # independent per token
    od = od.masked_fill(drop, self.null_idx) # null = <pad>
```

A3 canvas + A1 clean clamp (training) — `Restorer.forward`:

```
if self.dst_token:
    # canvas = [dst, ori, v1, ...]
    xs = [torch.cat([x[-1:], x]) for x in xs]
    xs = [torch.cat([x, full(L - len(x), eos_idx)]) for x in xs] # pad tail with <end>
    ...
if self.clean_prefix and self.od_cond:
    # A1: condition tokens stay clean
    if self.dst_token and d_kept: x_t[k][0] = xs[k][0]
    if o_kept: x_t[k][o_pos] = xs[k][o_pos]
```

In-context clamp + CFG (sampling) — `Restorer.sample_with_len`:

```
if getattr(self, "clean_prefix", False):
    xt[:, :prefix_len] = prefix[:, :prefix_len] # held clean every step
else:
    prefix_t = self.destroyer.diffusion(prefix, ts) # base: re-noised replacement
    xt[:, :prefix_len] = prefix_t[:, :prefix_len]

x0_pred_logits = self.restore(xt, lengths, ts, od=od)
if self.od_cond and od is not None and w_cfg != 1.0: # classifier-free guidance
    x0_uncond_logits = self.restore(xt, lengths, ts, od=None)
    x0_pred_logits = x0_uncond_logits + w_cfg * (x0_pred_logits - x0_uncond_logits)
```

1.2 Losses currently in use

The training objective combines three always-on terms (inherited from the reference paper's design and the repo) and up to three optional terms added in this session. All terms are computed over the full 64-position canvas, so `<end>` positions are supervised exactly like vertices. The trainer combines them as

$$\mathcal{L} = \mathcal{L}_{\text{KL}} + \lambda_{\text{ce}} \mathcal{L}_{\text{CE}} + \lambda_{\text{con}} \mathcal{L}_{\text{con}} + \lambda_{\text{arr}} \mathcal{L}_{\text{arr}}, \quad \lambda_{\text{ce}} = \lambda_{\text{con}} = 1, \quad \lambda_{\text{arr}} \in \{0, 0.05\} \quad (1)$$

(Repo quirk: once $\mathcal{L}_{\text{CE}} < 3.75$ the trainer optimizes $\mathcal{L}_{\text{KL}}$ alone; the other terms keep improving as side effects. $\mathcal{L}_{\text{arr}}$ is folded into the reported KL value.)

Main ELBO term. Writing v_ℓ for the one-hot vector of the canvas token at position ℓ , $\hat{x}_0^\ell$ for the model's predicted x_0 distribution at position ℓ (a row vector over the $V+1$ states), and $C_t, \bar{C}_t$ for the one-step / cumulative CTMC kernels (Eq. 10, §7.2), the true and model posteriors are the reference paper's Eq. (7)/(13):

$$q(x_{t-1}^\ell | x_t^\ell, x_0^\ell) = \text{Cat}(x_{t-1}^\ell | p \propto v_t^\ell C_t \odot v_0^\ell \bar{C}_{t-1}), \quad p_\theta(x_{t-1}^\ell | x_t, c) = \text{Cat}(x_{t-1}^\ell | p \propto v_t^\ell C_t \odot \hat{x}_0^\ell \bar{C}_{t-1}) \quad (2)$$

$$\mathcal{L}_{\text{KL}} = \mathbb{E}_{x_0, t \sim \mathcal{U}[1, T]} \left[\frac{1}{\sum_\ell w_\ell} \sum_{\ell=1}^{64} w_\ell D_{\text{KL}}(q(x_{t-1}^\ell | x_t^\ell, x_0^\ell) \parallel p_\theta(x_{t-1}^\ell | x_t, c)) \right] \quad (3)$$

x_0 -prediction term — direct cross-entropy supervision of the denoiser head (trains `<end>` placement, i.e. length):

$$\mathcal{L}_{\text{CE}} = \mathbb{E}_{x_0, t} \left[\frac{1}{\sum_\ell w_\ell} \sum_{\ell=1}^{64} -w_\ell \log \hat{x}_0^\ell[x_0^\ell] \right] \quad (4)$$

Position weights (base model: all $w_\ell = 1$; A2/A4a modify them, with $s = 1$ if the dst token occupies position 0, else 0, and L the true path length):

$$w_\ell = \begin{cases} \lambda_{\text{eos}} & \text{if } x_0^\ell = \langle \text{end} \rangle \quad (\text{A2, } \lambda_{\text{eos}} = 0.1) \\ \lambda_{\text{dst}} & \text{if } \ell = s + L - 1 \quad (\text{A4a, endpoint position, } \lambda_{\text{dst}} = 5) \\ 1 & \text{otherwise} \end{cases} \quad (5)$$

Connectivity term — a mean-field graph-topology prior on consecutive $\hat{x}_0$ marginals ($\times 100$ scale in code): it rewards probability mass on adjacent vertex pairs so that $\hat{x}_0$ decodes into edge-valid routes:

$$\mathcal{L}_{\text{con}} = - \sum_{\ell=s}^{62} \left[\hat{x}_0^\ell (A \log \hat{x}_0^{\ell+1})^\top + \hat{x}_0^{\ell+1} (A \log \hat{x}_0^\ell)^\top \right] \quad (6)$$

Arrival auxiliary (A4b) — a differentiable surrogate of the arrival metric, same product-of-marginals family as $\mathcal{L}_{\text{con}}$: it maximizes the mean-field probability that the destination is immediately followed by $\langle \text{end} \rangle$ *somewhere* on the canvas (position-free, complementing the position-fixed A4a):

$$\mathcal{L}_{\text{arr}} = - \log \sum_{\ell=1}^{63} \hat{x}_0^\ell [D] \hat{x}_0^{\ell+1} [\langle \text{end} \rangle] \quad (7)$$

term	role	status
$\mathcal{L}_{\text{KL}}$ (3)	variational bound — makes one-step denoising match the true CTMC reverse dynamics; the term that makes the sampler correct	on (main)
$\mathcal{L}_{\text{CE}}$ (4)	sharpens $\hat{x}_0$ prediction; carries the length signal ($\langle \text{end} \rangle$ placement)	on ($\lambda_{\text{ce}} = 1$)
$\mathcal{L}_{\text{con}}$ (6)	graph-topology prior on consecutive marginals; under A3 the sum starts at $\ell = 1$	on ($\lambda_{\text{con}} = 1$)
A2: λ_{eos} (5)	rebalances supervision away from the trivial eos tail ($\sim 2/3$ of the canvas); risk: too low can miscalibrate length	queued (A1+A2+A3)
A4a: λ_{dst} (5)	emphasizes the one position that defines task success; with D in-context (A3) it becomes a well-posed copy task	queued (A1+A3+A4)
A4b: $\mathcal{L}_{\text{arr}}$ (7)	directly optimizes "D then stop" coupling anywhere on the canvas	queued (A1+A3+A4)

Not used: an RL-style reward on sampled arrival (non-differentiable, overlaps with the discriminator-guidance line of work) and CFG-specific losses (CFG needs no extra loss — condition dropout suffices, §3.0).

2. $\langle \text{end} \rangle$ -state degree: d = 0.05 vs d = 2.0 (base pair, final)

Both models identical except `eos_deg`. Conditional sampling (O prefix + OD embedding), CFG w = 1.

model / stage	valid	arrival	valid $\wedge$ arr	hit	EM	PC	len err	len corr	gen len
d=0.05 raw	1.000	0.010	0.010	0.125	0.003	0.034	37.7	-0.12	58.8
d=0.05 refine_es	0.990	0.186	0.186	0.186	0.327	0.514	9.2	—	—
d=2.0 raw	0.999	0.010	0.010	0.056	0.041	0.191	7.6	+0.44	18.5
d=2.0 refine_es	0.979	0.102	0.102	0.102	0.534	0.695	12.6	—	—

- d = 0.05 never learns to stop**: mean generated length 58.8 on a 64 canvas, len corr ≈ 0 . Its higher refine_es arrival (0.186) is an artifact — long wandering paths pass near the destination and refinement cuts them there.
- d = 2.0 learns a real, OD-conditioned length distribution**: len corr +0.44 with conditioning vs -0.13 without, mean length 18.5 vs real 22.9. Direct evidence that the $\langle \text{end} \rangle$ mechanism absorbs $p(L | O, D)$ into the model — the goal of removing the GMM.
- The prior block-diffusion finding (d = 0.05 best) does NOT transfer to full-canvas**: in a fixed 64-canvas where $\sim 2/3$ of training targets are $\langle \text{end} \rangle$, a too-small d starves the forward process of real $\rightarrow \langle \text{end} \rangle$ transitions and the model never emits $\langle \text{end} \rangle$. All ablations therefore build on d = 2.0.

3. Classifier-free guidance sweep (cond variant, both models, final)

3.0 Why CFG is available, and the formulation

Training side (condition dropout, in words). Before each training sample's condition is embedded, the origin token is replaced by the null token ($\langle \text{pad} \rangle$) with probability 0.1, and — independently — the destination token is replaced with probability 0.1. The null token simply means "this condition is not given". As a result, about 81% of samples are trained with both O and D, 9% with O only, 9% with D only, and 1% with no condition at all, so a **single network $p_\theta(x_0 | x_t, c)$ jointly learns the conditional and the unconditional distribution** (and the two partial ones). This is exactly what makes classifier-free guidance possible: CFG needs both $p_\theta(x_0 | x_t, c)$ and $p_\theta(x_0 | x_t)$ from the same model, and dropout training provides the latter for free.

Sampling side. At every reverse step the model is called twice — once with the condition, once with null tokens — and the per-position $\hat{x}_0$ logits are combined by extrapolation:

$$z_w = z_\emptyset + w(z_c - z_\emptyset) \quad \Longleftrightarrow \quad \tilde{p}_w(x_0 | x_t, c) \propto p_\theta(x_0 | x_t) \left[\frac{p_\theta(x_0 | x_t, c)}{p_\theta(x_0 | x_t)} \right]^w \quad (8)$$

By Bayes' rule the bracketed ratio equals (up to a c -only constant) the **implicit classifier** $p_\theta(c | x_0, x_t)$, so CFG samples from

$$\log \tilde{p}_w = \log p_\theta(x_0 | x_t) + w \log p_\theta(c | x_0, x_t) + \text{const} \quad (9)$$

— it sharpens generation toward states the implicit classifier attributes to the given O/D. Special cases: $w = 0 \rightarrow$ unconditional, $w = 1 \rightarrow$ the plain conditional model, $w > 1 \rightarrow$ classifier amplification (the regime tested below). In our discrete setting the guided logits z_w define the per-position $\hat{x}_0$ distribution, which then feeds the CTMC reverse posterior $q(x_{t-1} | x_t, \hat{x}_0)$ — guidance modifies *what the model believes the clean path is*, and the kernel handles the rest. Cost: $w \neq 1$ doubles the denoiser calls per step.

w	d = 0.05					d = 2.0				
	raw arr	ref arr	raw hit	EM raw/ref	PC raw/ref	raw arr	ref arr	raw hit	EM raw/ref	PC raw/ref
1.0	0.010	0.186	0.125	0.003 / 0.327	0.034 / 0.514	0.010	0.102	0.056	0.041 / 0.534	0.191 / 0.695
1.5	0.011	0.242	0.177	0.003 / 0.327	0.037 / 0.513	0.016	0.147	0.083	0.023 / 0.481	0.171 / 0.655
2.0	0.016	0.262	0.197	0.002 / 0.358	0.043 / 0.547	0.020	0.163	0.102	0.032 / 0.521	0.174 / 0.681
4.0	0.012	0.166	0.123	0.002 / 0.379	0.050 / 0.558	0.016	0.113	0.077	0.043 / 0.554	0.196 / 0.704

- **w = 2 is best**, roughly doubling hit/arrival over w = 1; w = 4 over-sharpens. This differs from the prior report (w = 1 best) because there the destination was also an in-context token — with embedding-only conditioning, guidance has real signal to amplify.
- Even at w = 2 the absolute level stays low (raw arrival 2%) — **guidance cannot rescue a conditioning channel that is too weak**; the fix must be architectural (§4).

3.1 Conditioning-channel ablation (d=2.0, w=1, raw)

variant	arrival	hit	len corr	gen len
cond (O prefix + OD emb)	0.010	0.056	+0.44	18.5
prefix_only (O prefix, null emb)	0.001	0.007	−0.13	11.9
uncond	0.001	0.006	−0.03	12.4

The OD embedding clearly works (10× on hit, restores length correlation) — it is just too weak to *steer* the trajectory onto the destination. Diagnosis: additive global conditioning through the time pathway is a weak steering signal in this conv UNet; the destination needs a position-anchored in-context representation → ablation A3.

4. Ablation: anchored conditioning (A1+A3) — final

Config (d = 2.0): **A1** = condition tokens held clean (never noised when active), identically in training and sampling; **A3** = canvas $[dst, ori, v_1, \dots]$ with the destination clamped in-context at position 0. Evaluated on CPU (1,000 OD pairs, same protocol as §2).

model / stage	valid	arrival	valid^arr	hit	EM	PC	len err	len corr	gen len
base d=2.0, w=1, raw	0.999	0.010	0.010	0.056	0.041	0.191	7.6	+0.44	18.5
A1+A3, w=1, raw	1.000	0.056	0.056	0.138	0.029	0.190	6.5	+0.56	19.7
A1+A3, w=2, raw	1.000	0.061	0.061	0.149	0.026	0.169	6.0	+0.62	21.1
base d=2.0, w=2, refine_es	0.976	0.163	0.163	0.163	0.521	0.681	12.5	—	—
A1+A3, w=1, refine_es	0.980	0.180	0.180	0.180	0.506	0.674	11.0	—	—
A1+A3, w=2, refine_es	0.986	0.211	0.211	0.211	0.481	0.661	10.4	—	—

- **Anchoring the destination in-context is the single most effective change so far**: raw arrival 0.010 → 0.056 (5.6×), hit 0.056 → 0.138, length correlation +0.44 → +0.62, mean length 18.5 → 21.1 (real 22.9). The conditioning-channel diagnosis of §3.1 is confirmed.
- **CFG becomes nearly redundant once D is in-context** (raw arrival 0.056 → 0.061 from w=1 → 2) — mirroring the prior block-diffusion report, where in-context conditioning made w = 1 optimal. The two conditioning channels are substitutes, and the in-context one is stronger.
- **Arrival decays sharply with OD distance** (raw, w=2): 12.4% for real length 2–15, 5.4% for 16–25, 2.7% for 26–35, 2.6% for 36+. Errors compound over long horizons in the mean-field full-canvas decode — the main remaining failure mode, and the strongest argument for the block-diffusion stage (commit blocks progressively instead of decoding 64 positions at once).
- Sanity: the same model run unconditionally arrives at chance level (0.001) and loses the length correlation (+0.05), so the gains are attributable to conditioning, not memorization.

4.1 A2 (eos loss weight) and A4 (destination losses) — final

Both retrained after the GPU recovery, stacked on A1+A3 (d = 2.0) for attribution: **A1+A2+A3** adds $\lambda_{eos} = 0.1$ (Eq. 5); **A1+A3+A4** adds $\lambda_{dst} = 5$ and $\lambda_{arr} = 0.05$ (Eqs. 5, 7). 1,000 OD pairs, best CFG weight per model shown alongside w = 1.

config	raw arrival	refine arrival	raw hit	EM raw/ref	PC raw/ref	len corr	gen len	len err
A1+A3, w=2 (§4 reference)	0.061	0.211	0.149	0.026 / 0.481	0.169 / 0.661	+0.62	21.1	6.0
A1+A2+A3, w=1	0.069	0.292	0.219	0.026 / 0.481	0.164 / 0.657	+0.43	21.8	7.7
A1+A2+A3, w=1.5 (best)	0.077	0.335	0.257	0.020 / 0.483	0.156 / 0.659	+0.47	23.7	8.0
A1+A3+A4, w=1	0.090	0.223	0.213	0.013 / 0.471	0.108 / 0.630	−0.36	32.7	22.2

EM/PC are essentially flat across the conditioning ablations (raw ≈ 0.01–0.03 / refined ≈ 0.47–0.48 and 0.66): they score each generated path against *its own* endpoints' shortest length, so they measure route economy rather than OD-planning success and are dominated by the refinement step. The arrival/hit columns remain the discriminating metrics.

- **A2 is confirmed**: rebalancing supervision away from the eos tail lifts refined arrival from 0.211 to **0.335 (+59%)** and hit from 0.149 to 0.257 — the best usable-path rate of the full-canvas track. Cost: per-pair length tracking softens (len corr +0.62 → +0.47) while the mean stays calibrated (23.7 vs real 22.9) — with only 10% weight on <end> targets the model keeps absolute length but tracks per-pair variation less tightly. Arrival improves in *every* length bin (e.g. 16–25: 0.054 → 0.080; 26–35: 0.027 → 0.046).
- **A4 is a partial success with the predicted formulation flaw confirmed**. It posts the best *raw* arrival (0.090, +48% over A1+A3), concentrated on short OD pairs (0.236 on length 2–15). But the arrival auxiliary as implemented maximizes the *expected count* of D→<end> patterns — the sum in Eq. 7 can

exceed 1, and training KL indeed went negative — so the model inflates length (gen 32.7 vs real 22.9, len corr **-0.36**) to fit more $D \rightarrow \langle \text{end} \rangle$ occurrences, and its unconditional mode collapses (valid 0.29, unique 0.30). The corrected "at-least-once" form, $\mathcal{L}_{\text{arr}} = -\log(1 - \prod_{\ell}(1 - \hat{x}_0^{\ell}[D] \hat{x}_0^{\ell+1}[\langle \text{end} \rangle]))$, is the obvious next iteration; it was not retrained here because the block-diffusion stage supersedes the full-canvas track.

- **Final full-canvas ranking (refined arrival):** $A1+A2+A3$ (0.335) > $A1+A3+A4$ (0.223) $\approx$ $A1+A3$ (0.211) >> base $d=2.0$ (0.163) >> soft-conditioning-only (0.102).

5. Interpretation

- **Length prediction works without a length model:** with a properly mixed $\langle \text{end} \rangle$ state ($d = 2.0$) the model learns $p(L | O, D)$ — len corr +0.44 base, +0.62 with anchored conditioning, ≈ 0 unconditionally. Conditioning, not data statistics, carries the length. The GMM is gone with no loss of calibration (mean length 21.1 vs real 22.9).
- **Conditioning strength was the primary bottleneck, and in-context anchoring is the right mechanism:** soft global embedding $\rightarrow \sim 1\%$ raw arrival; + tuned CFG $\rightarrow \sim 2\%$; + clean in-context D ($A1+A3$) $\rightarrow 5.6\text{--}6.1\%$, with CFG then nearly redundant. Ranking of channels: in-context token > CFG-amplified embedding > plain embedding.
- **The remaining gap is horizon compounding, not conditioning:** arrival is 12% for short OD pairs but $\sim 3\%$ beyond length 25. A 64-position mean-field canvas decoded in one pass accumulates too much error over long routes. This is precisely what the block-diffusion stage addresses (progressive block commitment; the prior report's mask-kernel block models reached 0.94+ arrival with the same in-context conditioning) — the present results de-risk its conditioning design: **destination as an always-clean in-context token + adaLN-style embedding, $d = 2.0$ -style well-mixed $\langle \text{end} \rangle$, no CFG needed.**
- **Loss rebalancing (A2) is the second confirmed lever** (+59% refined arrival on top of anchored conditioning), while **the A4 arrival auxiliary needs its at-least-once reformulation before reuse** — its count-maximizing form trades length calibration for short-range arrival (§4.1). For the block-diffusion stage: A2's imbalance does not arise by design (PAD blocks are excluded from the loss), and any arrival auxiliary should use the corrected form within the final block.
- `refine_es` remains a strong post-hoc booster but is graph surgery; model-internal arrival is the metric that must carry the improvement.

6. Self-directed changes and incidents during the session

- **Removed the external GMM length model** in favor of LLaDA-style $\langle \text{end} \rangle$ prediction (user direction); an earlier GMM-length training run was stopped and discarded per user instruction.
- **Parameter-registration fix:** base `EPSM` keeps UNet down/up blocks in plain Python lists — $\sim 1.79\text{M}$ of 2.81M parameters invisible to the optimizer. Fixed via `nn.ModuleList` for the new `EPSM_OD*` classes only (baseline left untouched for comparability, per user decision).
- **Sampling alignment fix:** `Restorer.sample()` sorted lengths but not the paths providing prefix/OD conditions; (O, D, L) triples could mix across samples. Paths are now sorted as units.
- **sync.sh exclude fix:** `sets_log / figs / sets_disc` added — `rsync --delete` was silently removing server-side logs.
- **d selection contra the prior report (§2) and CFG adoption at $w = 2$ (§3), both data-driven.**
- **GPU-0 hardware fault (~08:12):** GPU 0 dropped off the bus mid-training ("unspecified launch failure", then "Unable to determine the device handle for GPU0"), subsequently breaking CUDA initialization server-wide (driver-level; recovery needs root: `uvdm` module reload or reboot). Evidence it is not application code: a 99%-identical run completed 48.2k iters concurrently on GPU 1; the same GPU 0 had completed a full 1.9 h run earlier; user-space code cannot de-enumerate a device. **Mitigation:** evaluation switched to CPU (the 2.8M-param model samples fast enough), $A1+A2+A3$ retrain queued on GPU recovery.

7. Formulations

7.1 Canvas and losses

A path $p = (v_0=O, \dots, v_{|L|-1}=D)$ is embedded as $x = [v_0, \dots, v_{|L|-1}, \langle \text{end} \rangle, \dots, \langle \text{end} \rangle]$ (length 64); the A3 canvas prepends the destination: $x = [D, v_0, \dots]$. Objective: the repo's D3PM-style KL + CE + connectivity loss over the full canvas, with per-position weight λ_{eos} on $\langle \text{end} \rangle$ -target positions (A2; connectivity pairs start at position 1 for A3). Conditioning: $t \leftarrow \text{MLP}_t(t) + \text{MLP}_{od}([e(O); e(D)])$, null token $\langle \text{pad} \rangle$, independent dropout 0.1 $\rightarrow$ one model serves conditional / O-only / D-only / unconditional generation.

7.2 $\langle \text{end} \rangle$ as a CTMC state

With per-vertex weight $w = d/|V|$ (every real vertex connects to the $\langle \text{end} \rangle$ node with weight w , plus a w self-loop on $\langle \text{end} \rangle$):

$$A' = \begin{pmatrix} A & w\mathbf{1} \\ w\mathbf{1}^\top & w \end{pmatrix}, \quad G = A' - \text{diag}(\mathbf{1}^\top A'), \quad C_t = e^{G\beta_t}, \quad \bar{C}_t = C_1 C_2 \cdots C_t \quad (10)$$

A' symmetric $\Rightarrow$ columns of G sum to 0 and the uniform distribution over $|V|+1$ states remains the limiting distribution. d sets the forward leak rate $\text{real} \rightarrow \langle \text{end} \rangle$ and hence the reverse-process propensity to propose $\langle \text{end} \rangle$; the binarized A' additionally makes $\langle \text{end} \rangle$ a legal decode successor of every vertex and of itself (self-loop).

7.3 Sampling

Reverse diffusion from uniform noise over $V+1$ states, 10-sample MC posterior, adjacency-constrained left-to-right decode of $\hat{x}_0$, truncation at the first $\langle \text{end} \rangle$. Conditions: O (and D for A3) clamped — re-noised each step in the base pair, held clean under A1. CFG on $\hat{x}_0$ logits as formulated in §3.0.

8. Hyperparameters

flag	value	meaning
eos_mode / eos_canvas_len	on / 64	fixed canvas with predicted <end> (max real length 59)
eos_deg d	0.05 vs 2.0	total degree of the virtual <end> node (§2)
od_cond / od_dropout	on / 0.1	O/D embedding conditioning, independent per-token dropout
clean_prefix (A1)	ablation	condition tokens never noised when active (train = test)
eos_loss_weight (A2)	1.0 vs 0.1	loss weight on <end>-target positions
dst_token (A3)	ablation	canvas [dst, ori, ...]; dst clamped in-context
guidance_scale w	1–4 swept	CFG on $\times_0$ logits; w = 2 best (§3)
max_T / beta / dims / hidden	100 / 1e-4..10 / [100,120,200] / 32	unchanged repo defaults
bs / lr / epochs	32 / 5e-4 / 1 (48.2k iters)	training config, all runs

Artifacts

- Checkpoints: sets_model/0D_EOS_porto_v3_normal_d{0.05,2.0}.pth , ..._d2.0_A1A3.pth (..._d2.0_A1A2A3 pending retrain)
- Per-variant metrics: sets_res/od_eval/0D_EVAL_*.json (+ per-path EM/PC records, generated paths)
- Code: models_seq/eps_models.py (EPSM_OD, EPSM_OD_EOS), models_seq/seq_models.py (eos / A1 / A2 / A3 / CFG), eval_od_conditional.py , train_porto_od_eos*.sh